

LAPORAN PRAKTIKUM / TUGAS PEMROGRAMAN WEB

Pemrograman Web / Backend

Project CRUD, Dokumentasi API, dan Pengujian Sistem

Nama Mahasiswa	:	Kelompok 14
NIM	:	2418059, 2418010, 2418001, 2418066
Mata Kuliah	:	Web Service
Dosen Pengampu	:	Ahmad Faisol, ST., MT.
Semester	:	Semester Genap 2025/2026
Tanggal Pengumpulan	:	15 Juni 2026



INSTITUT TEKNOLOGI NASIONAL MALANG

Fakultas Teknologi Industri | Program Studi Teknik Informatika

2026

BAB I - PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi telah memberikan banyak kemudahan dalam penyebaran informasi kepada masyarakat, termasuk dalam bidang pariwisata. Informasi mengenai tempat wisata yang menarik, lokasi, deskripsi, serta kategori wisata perlu dikelola dengan baik agar dapat diakses secara mudah dan cepat oleh pengguna. Oleh karena itu, diperlukan sebuah sistem informasi wisata yang mampu mengelola data wisata secara terstruktur dan efisien.

Pada pengelolaan data wisata secara manual, sering ditemukan berbagai permasalahan seperti kesulitan dalam menambah, memperbarui, mencari, dan menghapus data wisata. Selain itu, data yang tidak tersimpan secara terpusat dapat menyebabkan terjadinya duplikasi informasi serta kesalahan dalam pengelolaan data. Permasalahan tersebut dapat menghambat penyampaian informasi wisata kepada masyarakat.

Berdasarkan permasalahan tersebut, dibuatlah project Sistem Informasi Wisata berbasis web yang menerapkan konsep CRUD (Create, Read, Update, Delete). Sistem ini memungkinkan administrator untuk mengelola data wisata dan kategori wisata secara lebih mudah melalui API yang terintegrasi dengan database. Dengan adanya sistem ini, proses pengelolaan informasi wisata menjadi lebih efektif, terorganisir, dan mudah dikembangkan di masa mendatang.

1.2 Tujuan

Tujuan dari pembuatan project dan laporan ini adalah:

- Membangun aplikasi Sistem Informasi Wisata yang menerapkan fungsi CRUD (Create, Read, Update, Delete) secara lengkap.
- Mengembangkan REST API untuk mengelola data wisata dan kategori wisata menggunakan teknologi web modern.
- Mendokumentasikan seluruh endpoint API yang tersedia dalam aplikasi.
- Melakukan pengujian sistem untuk memastikan seluruh fungsi berjalan dengan baik dan sesuai kebutuhan.
- Mengunggah source code dan dokumentasi project ke repositori GitHub sebagai bentuk pengelolaan versi dan dokumentasi proyek.

1.3 Ruang Lingkup

Ruang lingkup pekerjaan dalam tugas ini meliputi:

- Pengembangan aplikasi Sistem Informasi Wisata berbasis web.
- Pengelolaan data wisata dan kategori wisata menggunakan operasi CRUD.
- Pembuatan REST API untuk akses dan manipulasi data.
- Pembuatan dokumentasi spesifikasi API (API Specification).
- Pelaksanaan dan dokumentasi pengujian sistem menggunakan Postman.
- Pengelolaan database menggunakan MySQL dan Prisma ORM.
- Upload source code dan dokumentasi ke repositori GitHub.

1.4 Teknologi yang Digunakan

Backend	:	Node.js + Express.js v5.2.1
ORM	:	Prisma v7.8.0
Database	:	MySQL
Autentikasi	:	JSON Web Token
Validasi	:	Joi v18.2.1
Password Hashing	:	bcrypt v6.0.0
Slug Generator	:	slugify v1.6.9
Environment	:	dotenv v16.4.5
API Tool	:	Postman
Version Control	:	Git & GitHub

BAB II - PROJECT CRUD

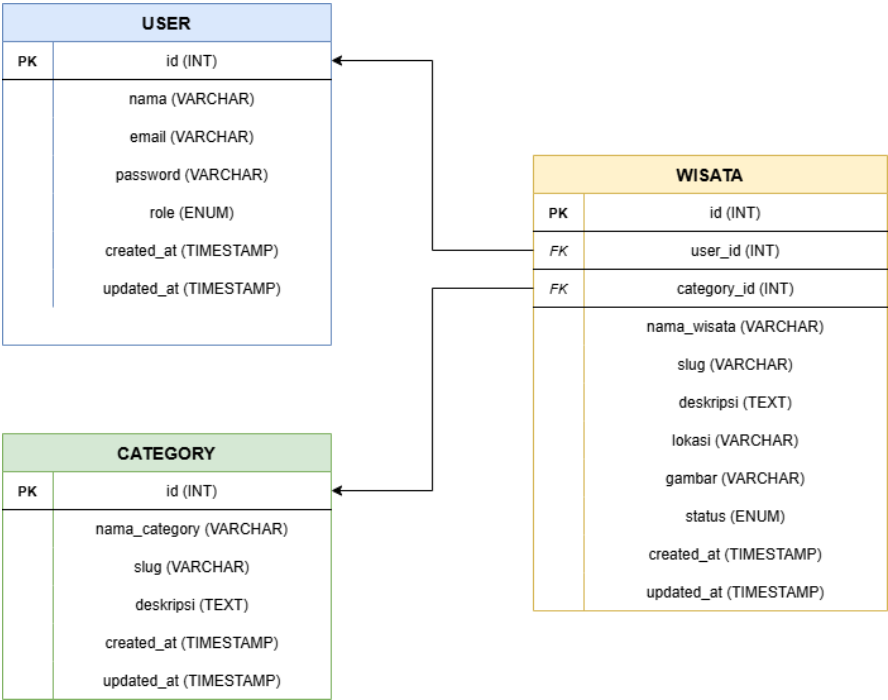
2.1 Deskripsi Project

Aplikasi ini merupakan News Portal API, sebuah sistem backend RESTful API untuk manajemen portal berita. API ini memungkinkan pengguna terautentikasi melakukan operasi CRUD terhadap dua entitas utama: Berita (news) dan Kategori (categories). Dibangun di atas Node.js + Express.js dengan Prisma ORM untuk interaksi ke database MySQL.

Seluruh endpoint (kecuali login) dilindungi middleware autentikasi JWT. Setiap berita memiliki judul, slug unik yang digenerate otomatis dari judul + timestamp, konten, gambar, serta relasi ke kategori dan pengguna.

2.2 Struktur Database

2.2.1 Relasi Antar Tabel



Gambar 2.1 Relasi Antar Tabel

Struktur database ini dirancang agar pengelolaan data wisata dapat dilakukan secara terorganisir, menghindari redundansi data, serta mempermudah proses pencarian dan pengelompokan informasi wisata.

2.2.2 Struktur Tabel

Tabel users:

Nama Field	Tipe Data	Constraint	Keterangan
id	INTEGER	PK, AUTO_INCREMENT, NOT NULL	ID unik pengguna
name	VARCHAR(191)	NOT NULL	Nama lengkap pengguna
email	VARCHAR(191)	UNIQUE, NOT NULL	Email pengguna
password	VARCHAR(191)	NOT NULL	Password pengguna
created_at	TIMESTAMP	NULLABLE*	Waktu dibuat
updated_at	TIMESTAMP	NULLABLE*	Waktu diperbarui

Tabel categories:

Nama Field	Tipe Data	Constraint	Keterangan
id	INTEGER	PK, AUTO_INCREMENT, NOT NULL	ID kategori
name	VARCHAR(191)	NOT NULL	Nama kategori wisata

Tabel wisata

Nama Field	Tipe Data	Constraint	Keterangan
id	INTEGER	PK, AUTO_INCREMENT, NOT NULL	ID wisata
nama	VARCHAR(191)	NOT NULL	Nama tempat wisata
lokasi	VARCHAR(191)	NOT NULL	Lokasi wisata
deskripsi	VARCHAR(191)	NOT NULL	Deskripsi wisata
gambar	VARCHAR(191)	NOT NULL	Path/URL gambar wisata

user_id	INTEGER	FK, NOT NULL	ID pengguna yang menambahkan wisata
category_id	INTEGER	FK, NOT NULL	ID kategori wisata

2.3 Fitur CRUD

2.3.1 Create (Tambah Data)

- **POST /api/categories:** Menambahkan data kategori wisata baru ke dalam database. Data yang dikirim berupa nama kategori. Sistem akan melakukan validasi untuk memastikan data yang diberikan tidak kosong sebelum disimpan.
- **POST /api/wisata:** Menambahkan data wisata baru ke dalam database. Data yang dikirim meliputi nama wisata, lokasi, deskripsi, gambar, user_id, dan category_id. Sistem akan memvalidasi seluruh data yang diperlukan sebelum proses penyimpanan dilakukan. Setelah berhasil ditambahkan, sistem mengembalikan data wisata yang baru dibuat beserta informasi relasinya.

2.3.2 Read (Tampil Data)

- **GET /api/categories:** Mengambil seluruh data kategori wisata yang tersedia.
- **GET /api/categories/:id:** Mengambil satu data kategori berdasarkan ID. Sistem mengembalikan status **404 Not Found** apabila data kategori tidak ditemukan.
- **GET /api/wisata:** Mengambil seluruh data wisata beserta data relasi **Category** dan **User** menggunakan fitur **include** pada Prisma.
- **GET /api/wisata/:id:** Mengambil satu data wisata berdasarkan ID. Sistem mengembalikan status **404 Not Found** apabila data wisata tidak ditemukan.

2.3.3 Update (Edit Data)

- **PUT /api/categories/:id:** Memperbarui data kategori wisata. Sistem akan memeriksa terlebih dahulu apakah kategori yang akan diperbarui tersedia di database.
- **PUT /api/wisata/:id:** Memperbarui data wisata berdasarkan ID. Data yang dapat diperbarui meliputi nama wisata, lokasi, deskripsi, gambar, kategori, maupun pengguna yang terkait. Sistem akan memeriksa keberadaan data sebelum melakukan proses pembaruan.

2.3.4 Delete (Hapus Data)

- **DELETE /api/categories/:id:** Menghapus kategori wisata berdasarkan ID. Sistem akan memeriksa keberadaan kategori sebelum melakukan penghapusan.

- **DELETE /api/wisata/:id:** Menghapus data wisata berdasarkan ID. Sistem akan memeriksa terlebih dahulu apakah data tersedia. Jika berhasil, sistem mengembalikan status **200 OK** beserta informasi data yang telah dihapus.

BAB III - DOKUMENTASI SPESIFIKASI API

3.1 Lingkungan Pengujian

Tools Pengujian : Postman

Lingkungan : Local

Database : MySQL 8.0

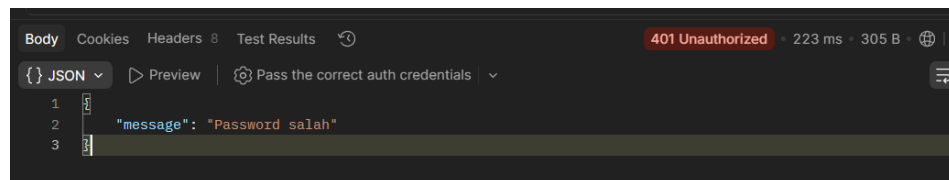
Penguji : Kelompok 14

3.2 Pengujian

3.2.1 Respons Sukses

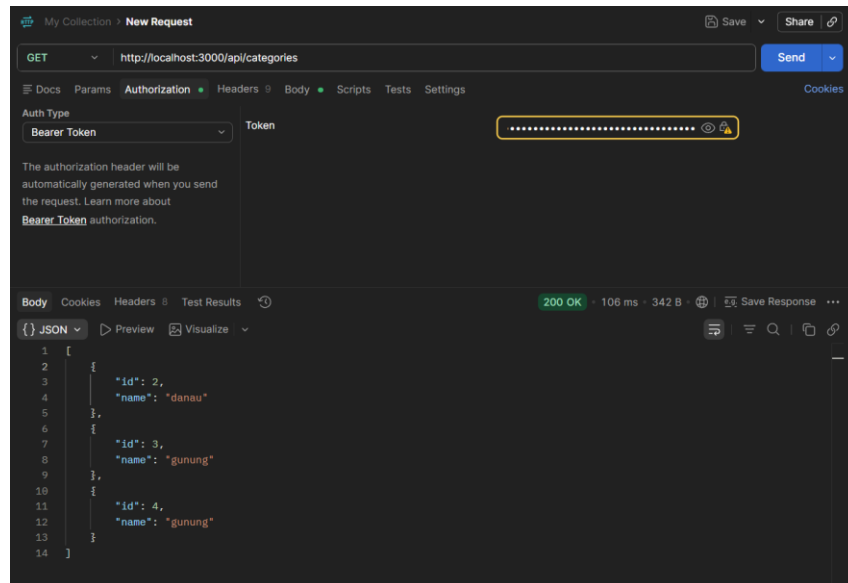


3.2.2 Respons Error

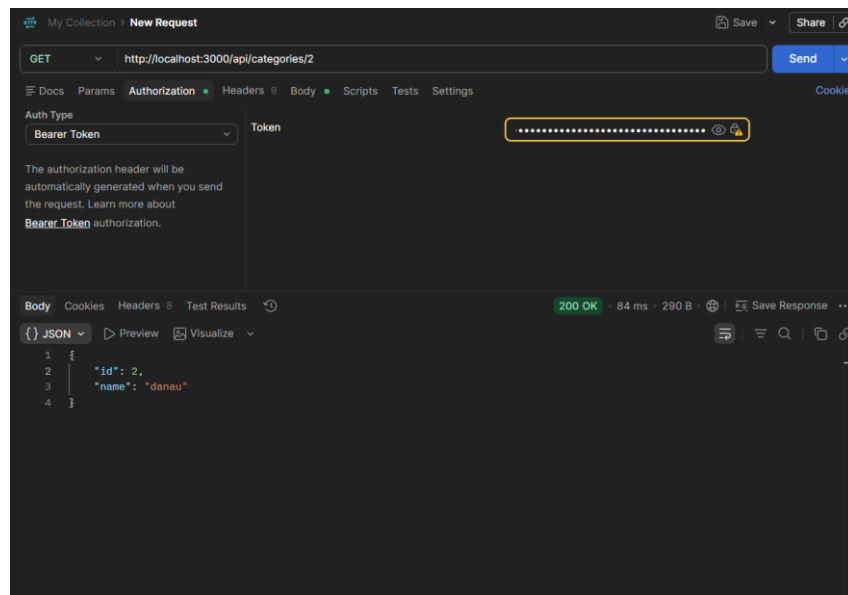


3.3 Daftar Endpoint

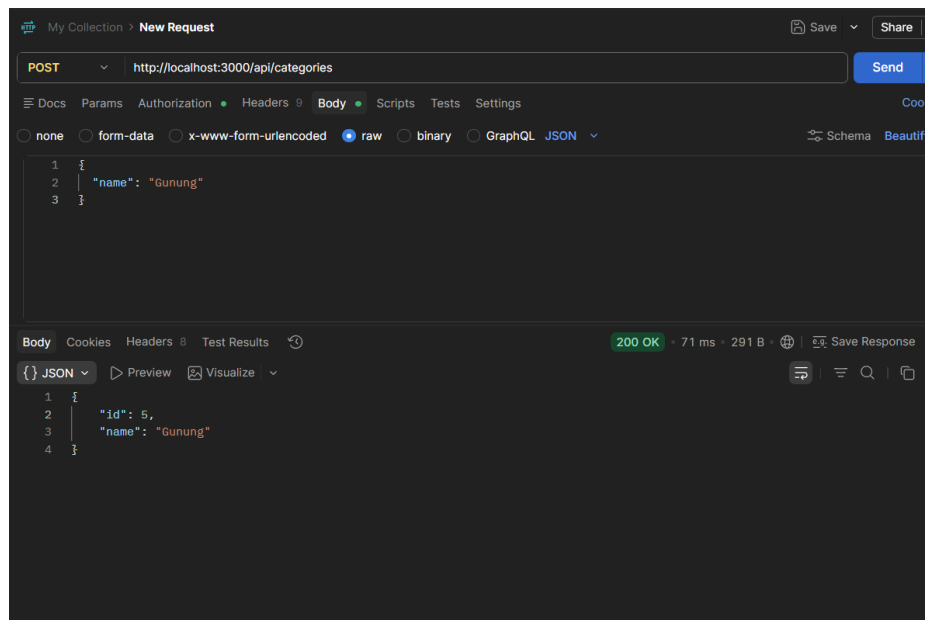
TEST GET CATEGORY:



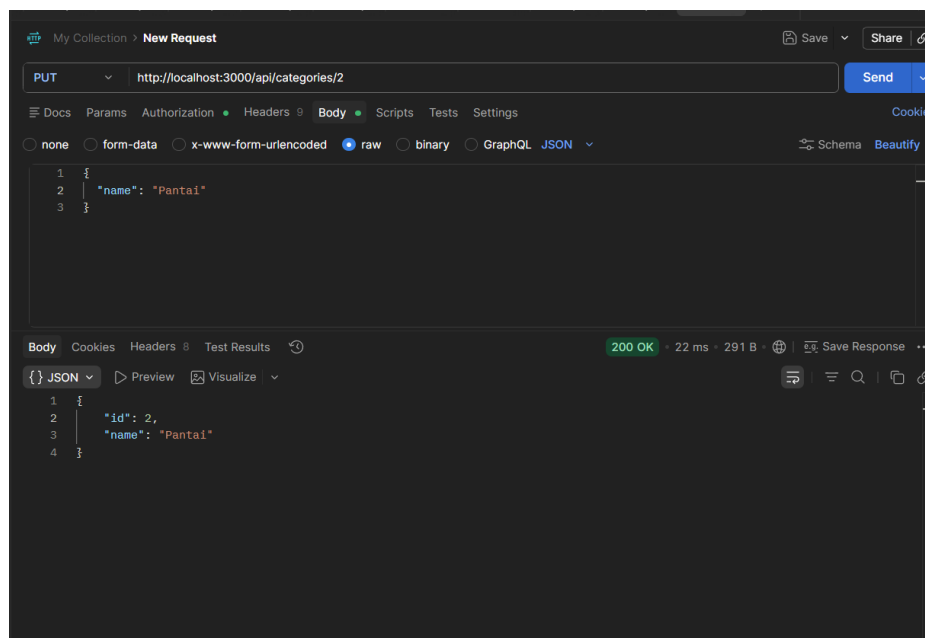
TEST GET CATEGORY BY ID:



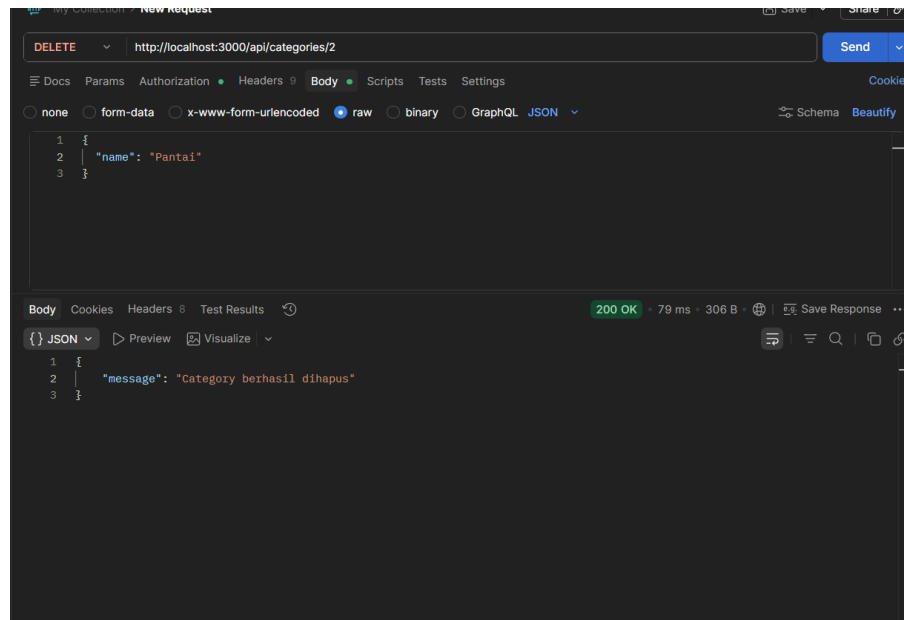
TEST POST CATEGORY:



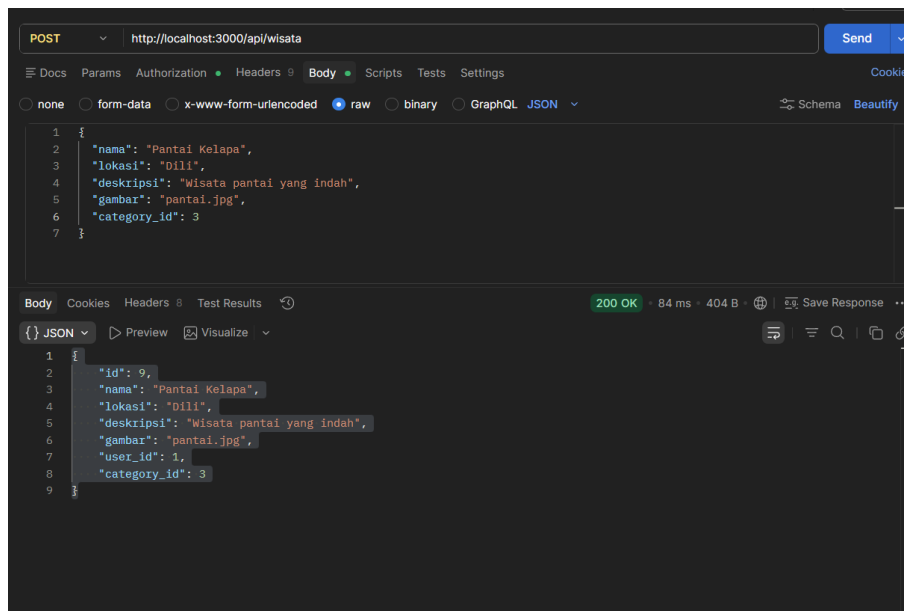
TEST PUT CATEGORY



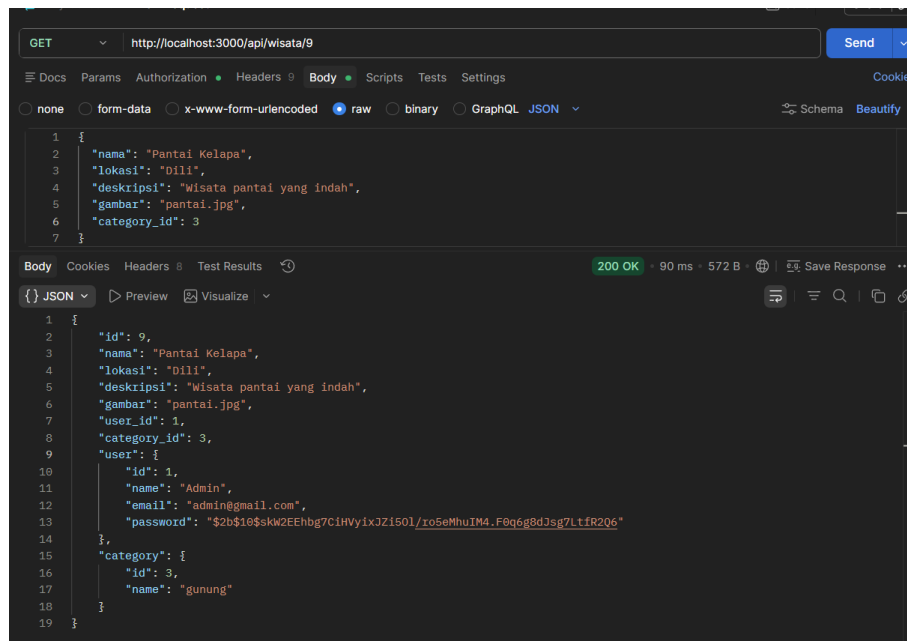
TEST DELETE CATEGORY



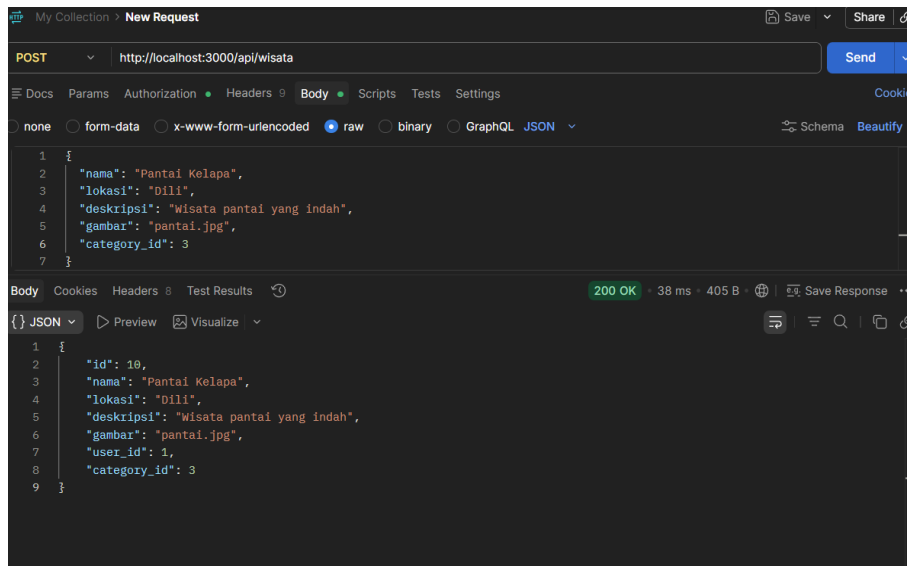
TEST GET WISATA



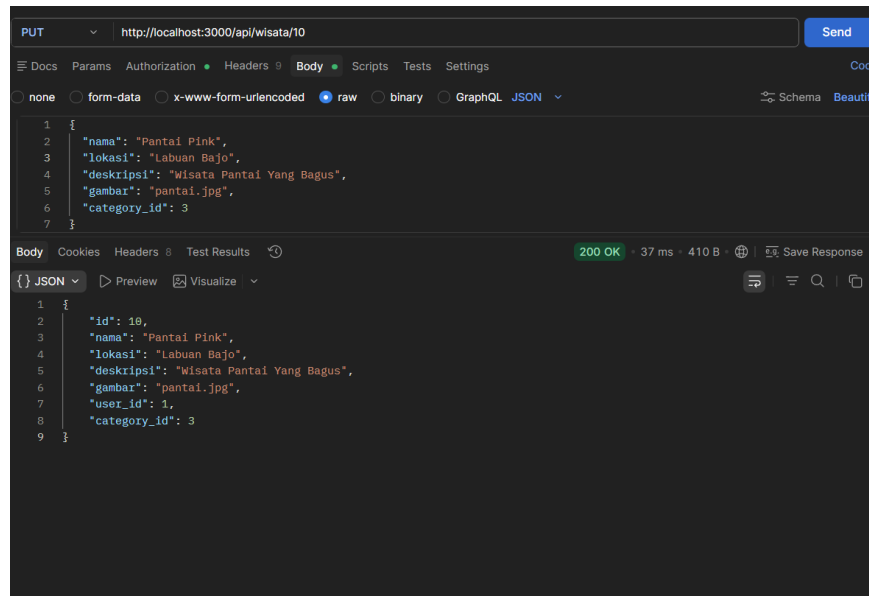
TEST GET WISATA BY ID



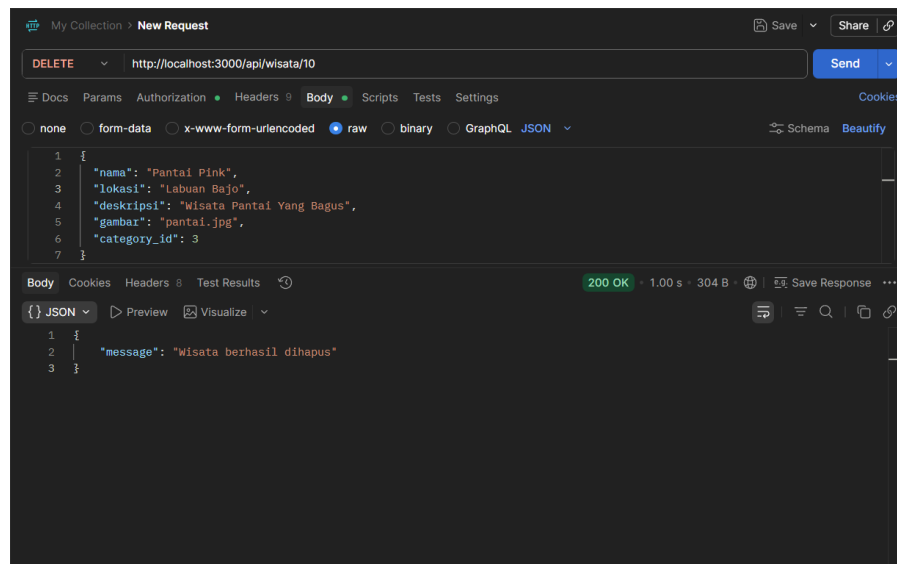
TEST POST WISATA



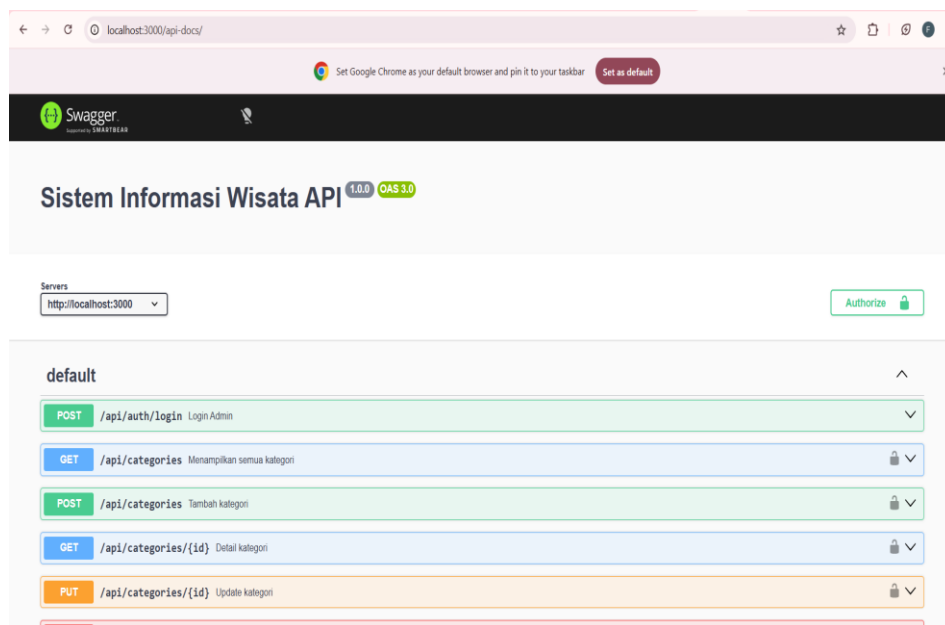
TEST PUT WISATA



TEST DELETE WISATA



TEST OPENAPI



BAB IV - DOKUMENTASI PENGUJIAN

4.1 Strategi Pengujian

Pengujian dilakukan secara manual menggunakan Postman dan Swagger UI untuk seluruh endpoint API. Setiap skenario mencakup kasus positif (input valid) dan kasus negatif (input tidak valid, ID tidak ditemukan, akses tanpa token, dan token tidak valid). Pengujian meliputi autentikasi JWT serta operasi CRUD pada entitas Category dan Wisata.

4.2 Lingkungan Pengujian

Tools Pengujian : Postman dan Swagger UI

Lingkungan : Local Development

Database : MySQL - db_wisata

Base URL : <http://localhost:3000/api>

Dokumentasi API : <http://localhost:3000/api-docs>

Tanggal Pengujian : 08 Juni 2026

Penguji : Kelompok 14

4.3 Skenario dan Hasil Pengujian

4.3.1 Pengujian Autentikasi

No	Skenario Pengujian	Data Input	Hasil yang Diharapkan	Status
1	Login dengan kredensial valid	email & password benar	200 OK + JWT Token	Berhasil
2	Login dengan email tidak terdaftar	email salah	404 User tidak ditemukan	Berhasil
3	Login dengan password salah	password salah	401 Password salah	Berhasil
4	Akses endpoint tanpa token	tanpa Authorization	401 Token tidak ada	Berhasil
5	Akses endpoint dengan token tidak valid	token acak	401 Token tidak valid	Berhasil

4.3.2 Pengujian Create

No	Skenario Pengujian	Data Input	Hasil yang Diharapkan	Status
1	POST /api/categories	name valid	201 Created	Berhasil
2	POST /api/categories	name kosong	400 Bad Request	Berhasil
3	POST /api/categories	tanpa token	401 Unauthorized	Berhasil
4	POST /api/wisata	data lengkap valid	201 Created	Berhasil
5	POST /api/wisata	category_id tidak ada	400 Bad Request	Berhasil
6	POST /api/wisata	category_id tidak ditemukan	400 Bad Request	Berhasil

4.3.3 Pengujian Read

No	Skenario Pengujian	Data Input	Hasil yang Diharapkan	Status
1	GET /api/categories	-	200 OK	Berhasil
2	GET /api/categories/{id}	id valid	200 OK	Berhasil
3	GET /api/categories/{id}	id tidak ditemukan	404 Data tidak ditemukan	Berhasil
4	GET /api/categories/{id}	id bukan angka	400 Bad Request	Berhasil
5	GET /api/wisata	-	200 OK	Berhasil
6	GET /api/wisata/{id}	id valid	200 OK	Berhasil
7	GET /api/wisata/{id}	id tidak ditemukan	404 Data tidak ditemukan	Berhasil

4.3.4 Pengujian Update

No	Skenario Pengujian	Data Input	Hasil yang Diharapkan	Status
1	PUT /api/categories/{id}	data valid	200 OK	Berhasil
2	PUT /api/categories/{id}	id tidak ditemukan	404 Data tidak ditemukan	Berhasil
3	PUT /api/categories/{id}	name kosong	400 Bad Request	Berhasil
4	PUT /api/wisata/{id}	data valid	200 OK	Berhasil

5	PUT /api/wisata/{id}	id tidak ditemukan	404 Data tidak ditemukan	Berhasil
6	PUT /api/wisata/{id}	data tidak lengkap	400 Bad Request	Berhasil

4.3.5 Pengujian Delete

No	Skenario Pengujian	Data Input	Hasil yang Diharapkan	Status
1	DELETE /api/categories/{id}	id valid	200 OK	Berhasil
2	DELETE /api/categories/{id}	id tidak ditemukan	404 Data tidak ditemukan	Berhasil
3	DELETE /api/categories/{id}	tanpa token	401 Unauthorized	Berhasil
4	DELETE /api/wisata/{id}	id valid	200 OK	Berhasil
5	DELETE /api/wisata/{id}	id tidak ditemukan	404 Data tidak ditemukan	Berhasil
6	DELETE /api/wisata/{id}	tanpa token	401 Unauthorized	Berhasil

4.4 Ringkasan Hasil Pengujian

Kategori Pengujian	Total TC	Berhasil	Gagal
Autentikasi	5	5	0
Create	6	6	0
Read	7	7	0
Update	6	6	0
Delete	6	6	0
TOTAL	30	30	0

BAB VI - PENUTUP

6.1 Kesimpulan

1. Sistem Informasi Wisata API berhasil dibangun menggunakan Express.js sebagai framework backend dan Prisma ORM sebagai penghubung antara aplikasi dengan database MySQL.
2. Sistem menyediakan operasi CRUD (Create, Read, Update, Delete) secara lengkap untuk entitas Category dan Wisata sehingga data dapat dikelola dengan baik.
3. Sistem autentikasi berbasis JSON Web Token (JWT) berhasil diimplementasikan dan mampu melindungi endpoint API dari akses yang tidak sah.
4. Prisma ORM berhasil diintegrasikan dengan database MySQL serta mendukung relasi antar tabel user, category, dan wisata.
5. Dokumentasi API berhasil dibuat menggunakan OpenAPI (Swagger) sehingga memudahkan pengembang dalam memahami, menguji, dan menggunakan endpoint yang tersedia.
6. Pengujian menggunakan Postman dan Swagger UI menunjukkan bahwa seluruh fungsi sistem berjalan sesuai dengan kebutuhan yang telah dirancang.
7. Berdasarkan hasil pengujian, seluruh skenario yang diuji berhasil dijalankan dengan tingkat keberhasilan mencapai 100%, sehingga Sistem Informasi Wisata API dinyatakan berjalan dengan baik dan memenuhi kebutuhan fungsional sistem.

6.2 Saran

1. Menambahkan fitur upload gambar secara langsung untuk data wisata sehingga pengguna tidak perlu memasukkan nama file gambar secara manual.
2. Mengimplementasikan fitur pencarian (search) dan filter berdasarkan kategori wisata untuk mempermudah pengguna menemukan informasi wisata yang diinginkan.
3. Menambahkan fitur pagination pada endpoint GET /api/wisata dan GET /api/categories agar performa sistem tetap optimal ketika jumlah data semakin banyak.
4. Mengimplementasikan Refresh Token untuk meningkatkan keamanan autentikasi dan manajemen sesi pengguna.
5. Menambahkan Role Based Access Control (RBAC) sehingga hak akses Admin dan User dapat dibedakan dengan lebih jelas.
6. Mengembangkan fitur pengelolaan data pengguna (User Management) secara lengkap melalui endpoint API.

7. Melakukan deployment aplikasi ke cloud server seperti Railway, Render, VPS, atau layanan cloud lainnya sehingga API dapat diakses secara online.
8. Menambahkan fitur rating dan ulasan wisata agar pengguna dapat memberikan penilaian terhadap tempat wisata yang tersedia dalam sistem.

DAFTAR PUSTAKA

Express.js. (2025). Express - Node.js Web Application Framework. Diakses pada 8 Juni 2026 dari <https://expressjs.com/>

JWT.io. (2025). JSON Web Tokens Introduction. Diakses pada 8 Juni 2026 dari <https://jwt.io/>

MySQL. (2025). MySQL Documentation. Oracle Corporation. Diakses pada 8 Juni 2026 dari <https://dev.mysql.com/doc/>

Node.js. (2025). Node.js Documentation. OpenJS Foundation. Diakses pada 8 Juni 2026 dari <https://nodejs.org/>

OpenAPI Initiative. (2025). OpenAPI Specification. Diakses pada 8 Juni 2026 dari <https://swagger.io/specification/>

Postman. (2025). Postman API Platform Documentation. Diakses pada 8 Juni 2026 dari <https://learning.postman.com/>

Prisma. (2025). Prisma ORM Documentation. Diakses pada 8 Juni 2026 dari <https://www.prisma.io/docs>

Swagger. (2025). Swagger UI Documentation. Diakses pada 8 Juni 2026 dari <https://swagger.io/tools/swagger-ui/>

W3Schools. (2025). Node.js Tutorial. Diakses pada 8 Juni 2026 dari <https://www.w3schools.com/nodejs/>

Mozilla Developer Network (MDN). (2025). HTTP Request Methods. Diakses pada 8 Juni 2026 dari <https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods>